Contents

1 从 word2v 到 Transformer **2**
1.1 LSTM . . . 2
1.2 fasttext&word2vec . . . 2
1.3 BPE/WordPiece 分词 . . . 2
1.4 Transformer 原理 . . . 2
1.4.1 为什么层次化 softmax 没人用了 . . . 3
1.4.2 Transformer 会不会规划未来 . . . 3
1.5 Transformer 的 FLOPS 和访存带宽 . . . 3
1.5.1 attention 的 FLOPS . . . 3
1.5.2 FFN 的 FLOPS . . . 4
1.5.3 DIN 的 FLOPS . . . 4
1.5.4 Transformer 的访存 . . . 5
1.6 Transformer 加速 . . . 7
1.6.1 lightseq . . . 7

2 PLM：仅编码器/仅解码器/编码器 + 解码器 **7**
2.1 仅编码器的 BERT . . . 7
2.1.1 multi-head att 实现 . . . 8
2.1.2 masked-language-model 的实现 . . . 10
2.1.3 BERT 的可解释性 . . . 12
2.2 更复杂的 BERT . . . 12
2.2.1 中文 BERT . . . 12
2.2.1.1 WWM . . . 12
2.2.1.2 ERNIE . . . 12
2.2.2 跨语言 . . . 13
2.2.2.1 XLM . . . 13
2.2.3 更长序列 . . . 14
2.2.3.1 transformer-xl . . . 14
2.2.3.2 XLNet . . . 15
2.2.4 更多的任务 . . . 16
2.2.4.1 MT-DNN . . . 16
2.2.5 其他改进 . . . 16
2.2.5.1 RoBERTa . . . 16
2.2.5.2 DeBERTa . . . 16
2.2.5.3 ELECTRA . . . 16
2.2.5.4 Matryoshka . . . 17
2.2.5.5 Piccolo2 . . . 19
2.2.5.6 Conan-Embedding . . . 19
2.2.5.7 MEXMA . . . 20
2.2.5.8 ModernBert . . . 20
2.3 更小的 BERT . . . 20
2.3.1 albert . . . 20
2.3.2 distillbert . . . 21
2.3.3 tinybert . . . 21
2.3.4 reformer . . . 22
2.3.5 LTD-bert . . . 22
2.3.6 Q-bert . . . 22
2.3.7 Adabert . . . 22
2.4 仅解码器的 GPT . . . 22
2.4.1 GPT . . . 22
2.4.2 GPT2 . . . 23
2.4.3 nanogpt 代码解读 . . . 23

2.5 编码器 + 解码器 25
2.5.1 T5 25
2.5.2 MASS 26
2.5.3 BART 26

1 从 word2v 到 Transformer

1.1 LSTM

超生动图解 LSTM 和 GRU，一文读懂循环神经网络！

1.2 fasttext&word2vec

注：w2v 训练时的内积不是 2 个 emb-in 的内积，而是 emb-in 和 emb-out 的内积

fasttext 源码解析

- `Dictionary::readWord`：空格分割，一次读出来一个 word
- `Dictionary::add`：每个 word 求个 hash，加进词典时，id 就是从 0 开始的序号，同时记录一下词频
- `Dictionary::threshold`：按词频排序，扔掉低频词
- `Dictionary::initNgrams`：每个词，加上前缀 BOW（<）和后缀（>），然后先扔进这个词的 subwords 里，然后再调用 `Dictionary::computeSubwords` 把这个词的 ngrams 也扔进它的 subwords 里

整个词表，是 word 数 +bucket 这么大，其中 bucket 表示可容纳的 subwords 和 wordNgrams 的数量，默认 200w

为什么 Word2Vec 训练中, 需要对负采样权重开 3/4 次幂?

Distributed Representations of Words and Phrases and their Compositionality里提到

is not important for our application.

Both NCE and NEG have the noise distribution $P_n(w)$ as a free parameter. We investigated a number of choices for $P_n(w)$ and found that the unigram distribution $U(w)$ raised to the 3/4rd power (i.e., $U(w)^{3/4}/Z$) outperformed significantly the unigram and the uniform distributions, for both NCE and NEG on every task we tried including language modeling (not reported here).

2.3 Subsampling of Frequent Words

通过对权重开 3/4 次幂，可以提升低频词被抽到的概率。在保证高频词容易被抽到的大方向下，通过权重 3/4 次幂的方式，适当提升低频词、罕见词被抽到的概率。如果不这么做，低频词，罕见词很难被抽到，以至于不被更新到对应的 Embedding。

1.3 BPE/WordPiece 分词

【Subword】深入理解 NLP Subword 算法：BPE、WordPiece、ULM

1.4 Transformer 原理

从三大顶会论文看百变 Self-Attention

包学包会，这些动图和代码让你一次读懂「自注意力」

http://jalammar.github.io/illustrated-transformer/

从熵不变性看 Attention 的 Scale 操作

Transformers Assemble（PART I） 讲了 3 篇

Transformers Assemble（PART II） 又讲了三篇

1.4.1 为什么层次化 softmax 没人用了

Transformer 结构中最后一层 softmax 为什么不再使用层次化 softmax 了呢?

主要还是计算资源的问题。

Mikolov 发明 word2vec 的几个版本大概在 13-14 年前后。那个时候 GPU 非常少见，印象里面 CMU 的 NLP 组没有 GPU，Stanford NLP lab 只有 6 块 K40。

大规模直接算 softmax 是在 google 的 14 年那篇 seq2seq 做 MT 的文章。为了快，把一个 softmax 并行在 4 块 GPU 上，每个 GPU 负责四分之一。那个年代，大多数 NLP 组全组都不会有 4 块 GPU。

hierarchical softmax 是 softmax 的近似，suboptimal 的。当如今计算资源足够大的时候，当然包括时间和显存 (BERT 和 Elmo 都没有用 hierarchical)，hierarchical softmax 就逐渐退出了历史舞台。

1.4.2 Transformer 会不会规划未来

Transformer 本可以深谋远虑，但就是不做

Do Language Models Plan for Future Tokens?

在训练期间的梯度既会为**当前 token 位置**的损失优化权重，也会为该序列**后面的 token** 进行优化，那么这二者会以怎样的比例分配资源?

- 预缓存假设（pre-caching hypothesis）：在时间步 t 计算与**当前时间步的推理任务无关**但**可能对未来时间步 $t+\tau$ 有用**的特征
- 面包屑假设（breadcrumbs hypothesis）：与时间步 t **最相关的特征**已经**等同**于将在**时间步 $t+\tau$ 最有用**的特征。

设计了一种合成场景，其中**只能通过显式的预缓存完成任务**，即模型必须为下一 token 预先计算信息，**否则就无法在一次单向通过中准确计算出正确答案**。发现明显的证据说明 transformer 可以学习预缓存，即当必须预计算信息来最小化损失时，它们就会这样做。

但在真实语言数据上，语言模型并不会显著地准备用于未来的信息。相反，它们是计算对预测**下一个 token 有用的特征**——事实证明**这对未来的步骤也很有用**。

1.5 Transformer 的 FLOPS 和访存带宽

https://zhuanlan.zhihu.com/p/624740065

A 的 shape 是 $m \times k$，B 的 shape 是 $k \times n$，那么矩阵乘法 AB 需要 $m \times k \times n$ 次的乘法，也需要同样多次的加法，所以 FLOPS 是 $2 \times m \times k \times n$

假设 batchsize 是 b，序列长度 s，原来的 emb 是 d，即输入的是 $[b, s, d]$，一般 $d = d_k = d_v = d_q$，W_Q、W_K、W_V 都是 $d_v \times d_v$，对应的 Q、K、V 矩阵都是 $s \times d_v$，有 $head_num$ 个头，每个头的维度 $per_head_d = \frac{d}{head_num}$

1.5.1 attention 的 FLOPS

attention 的公式：

$$Q = xW_Q, K = xW_K, V = xW_V$$
$$x_{\text{out}} = \text{softmax}\left(\frac{QK^T}{\sqrt{h}}\right) \cdot V \cdot W_o + x$$

- 计算 3 个 Q、K、V：要算三次 $s \times d$ 和 $d \times d_v$ 的矩阵乘法，所以是：$3 \times 2 \times b \times s \times d \times d_v$
  - 输入：$[b, s, d]$ 和 3 个 $[b, d, d_v]$
  - 输出：$[b, s, d_v]$，再把最后一维 d_v 拆成 $head_num$ 份，再把中间两维互换一下，得到 $[b, head_num, s, per_head_d]$
- 计算 Q 和 K 的相似度：要算一次 $s \times d_v$ 和 $d_v \times s$ 的矩阵乘法，$2 \times b \times s^2 \times d_k$
  - 输入：$[b, head_num, s, per_head_d]$ 和 $[b, head_num, per_head_d, s]$
  - 输出：$[b, head_num, s, s]$
- 把相似度用到 V 上：要算一次 $s \times s$ 和 $s \times d_v$ 的矩阵乘法，，$2 \times b \times s^2 \times d_v$
  - 输入：$[b, head_num, s, s]$ 和 $[b, head_num, s, per_head_d]$
  - 输出：$[b, head_num, s, per_head_d]$
- 最后过一个线性映射：要算一次 $s \times d_v$ 的和 $d_v \times d_v$ 的矩阵乘法，$2 \times b \times s \times d_v \times d_v$

- 输入：$[b, s, d_v]$ 和 $[d_v, d_v]$
- 输出：$[b, s, d_v]$

因为 $d_k = d_v = d_q = d$，单纯计算 attention 总共就是 $8bsd^2 + 4bs^2d$

1.5.2 FFN 的 FLOPS

FFN 的公式：

$$x = f_{\text{gelu}}\left(x_{\text{out}} W_1\right) W_2 + x_{\text{out}}$$

在原始 Transformer 中，W_1 的 shape 是 $[d, 4d]$，W_2 的 shape 是 $[4d, d]$，

- 第一个线性层：$2 \times b \times s \times\ d \times 4d = 8 \times b \times s \times\ d^2$
  - 输入：$[b, s, d]$ 和 $[d, 4d]$
  - 输出：$[b, s, 4d]$
- 第二个线性层：$2 \times b \times s \times\ 4d \times d = 8 \times b \times s \times\ d^2$
  - 输入：$[b, s, 4d]$ 和 $[4d, d]$
  - 输出：$[b, s, d]$

所以一层 Transformer，即 attention+FFN 的计算量为 $(8bsd^2 + 4bs^2d) + 16bsd^2 = 24bsd^2 + 4bs^2d$

有两点需要注意的：

- 对 NLP 任务来讲，一般 d 是个比较固定的值，如 512，而 s 变大，效果会更好，所以一般是 $s > d$，所以复杂度取决于 s 的大小。
- 但有些模型的初始设置不是这样的，例如 GPT3 的 175B 模型里，$s = 2048, d = 12288$，当然，算力够的话也可以把 s 变大

自己感觉：既然是 $24bsd^2 + 4bs^2d$，其实就是 $6sd^2$ 和 s^2d 的大小，即 $6d$ 和 s 的大小，如果 $6d > s$，则 d^2 起主导，反之 s^2 起主导

1.5.3 DIN 的 FLOPS

特殊地，对于推荐中的 DIN 那种，看当前 item 和历史 s 个 item 的相关性，即 q 的序列长度只有 1，不考虑多头，而这其实也是 decoder 预测下一个词时过一层 Transformer 的复杂度

已经有 3 个序列长度为 $s - 1$ 的 QKV 的 cache，要算第 s 个词和这 $s - 1$ 个词的 attention

- 计算第 s 个词的 3 个 Q、K、V：要算三次 $1 \times d$ 和 $d \times d_v$ 的矩阵乘法，所以是：$3 \times 2 \times b \times 1 \times d \times d_v$
  - 输入：$[b, 1, d]$ 和 3 个 $[b, d, d_v]$
  - 输出：$[b, 1, d_v]$
- 计算 Q 和 K 的相似度：要算一次 $1 \times d_v$ 和 $d_v \times s$ 的矩阵乘法，$2 \times b \times 1 \times d_k \times s$【这里的 K 是历史 $s - 1$ 长度的序列拼上当前词，当然对 DIN 来讲要去掉当前词，这里先忽略这个】
  - 输入：$[b, 1, d_v]$ 和 $[b, d_v, s]$
  - 输出：$[b, 1, s]$
- 把相似度用到 V 上：要算一次 $1 \times s$ 和 $s \times d_v$ 的矩阵乘法，，$2 \times b \times 1 \times d_v \times s$【同样地，这里的 V 是历史 $s - 1$ 长度的序列拼上当前词，当然对 DIN 来讲要去掉当前词，这里先忽略这个】
  - 输入：$[b, 1, s]$ 和 $[b, s, d_v]$
  - 输出：$[b, 1, d_v]$
- 最后过一个线性映射：要算一次 $1 \times d_v$ 的和 $d_v \times d_v$ 的矩阵乘法，$2 \times b \times 1 \times d_v \times d_v$
  - 输入：$[b, 1, d_v]$ 和 $[d_v, d_v]$
  - 输出：$[b, 1, d_v]$
- 第一个线性层：$2 \times b \times 1 \times\ d \times 4d = 8 \times b \times 1 \times\ d^2$
  - 输入：$[b, 1, d]$ 和 $[d, 4d]$
  - 输出：$[b, 1, 4d]$
- 第二个线性层：$2 \times b \times 1 \times\ 4d \times d = 8 \times b \times 1 \times\ d^2$
  - 输入：$[b, 1, 4d]$ 和 $[4d, d]$
  - 输出：$[b, 1, d]$

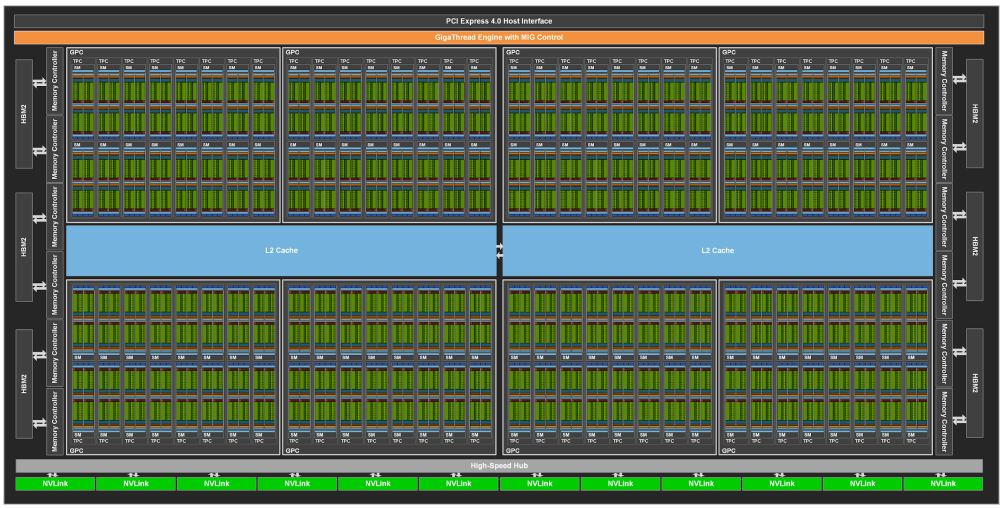
总共是 $6bd^2 + 2bds + 2bds + 2bd^2 + 8bd^2 + 8bd^2 = 24bd^2 + 4bds$

1.5.4 Transformer 的访存

GPU 架构的介绍参考https://developer.nvidia.com/blog/nvidia-ampere-architecture-in-depth/和https://www.zhihu.com/question/319355296/answer/2193938981，GPU 对比 CPU 如下：

- 任务模式
  - CPU 由专为顺序串行处理而优化的几个核心组成
  - GPU 则拥有一个由数以千计的更小、更高效的核心（专为同时处理多重任务而设计）组成的大规模并行计算架构。同时 CPU 相当的一部分时间在执行外设的中断、进程的切换等任务，而 GPU 有更多的时间并行计算。
- 功能定位
  - CPU 不但要承担计算任务还有承担逻辑控制等任务。
  - GPU 在渲染画面时需要同时渲染数以百万记的顶点或三角形，故 GPU 的设计是可以充分支持并行计算。
- 系统集成
  - GPU 作为一类外插设备，在尺寸、功率、散热、兼容性等方面的限制远远小于 CPU，这样可以让 GPU 有较大的显存和带宽。

以 A100 为例，整体架构如下



1. PCIE 层：通过 PCIE 接口以外设的方式集成到服务器上。
2. 中间一坨绿色的部分是 GPU 的计算核心 **SM(Streaming Multiprocessor)**，在 A100 中，一个 SM 有 64 个用于计算的 Core，共 108 个 SM(图里是 GA100, 有 128 个 SM)，故共 6192 个 Core。
3. 中间蓝色部分是 L2 缓存
4. NVLink：**多个 GPU 间进行通信**的组件，会优化 GPU 间的通信，提升传输效率。
5. 两侧的 HBM2 是显存，目前的 A100 的显存有两种 40G and 80G

A100 的 SM 如图所示，

GPU 的显存分成两部分：

- Global Memory：整体架构图中的**两侧 HBM2** 部分，例如 A100 80G 就有 80G 的 global memory，**2TB/s 带宽**，访问速度比较慢
- Shared Memory：SM 图中**浅蓝色**的 L1 Data Cache，例如 A100 中每个 SM 中有 **192KB**，访问速度比较快

从图中可见，A100 的 **FP16** 有 **312T 的 FLOPS**

Peak FP64[1]	9.7 TFLOPS
Peak FP64 Tensor Core[1]	19.5 TFLOPS
Peak FP32[1]	19.5 TFLOPS
Peak FP16[1]	78 TFLOPS
Peak BF16[1]	39 TFLOPS
Peak TF32 Tensor Core[1]	156 TFLOPS | 312 TFLOPS[2]
Peak FP16 Tensor Core[1]	312 TFLOPS | 624 TFLOPS[2]
Peak BF16 Tensor Core[1]	312 TFLOPS | 624 TFLOPS[2]
Peak INT8 Tensor Core[1]	624 TOPS | 1,248 TOPS[2]
Peak INT4 Tensor Core[1]	1,248 TOPS | 2,496 TOPS[2]

Table 1. A100 Tensor Core GPU performance specs.
1) Peak rates are based on the GPU boost clock.
2) Effective TFLOPS / TOPS using the new Sparsity feature.

以矩阵乘法为例，$[M,K]\times[K,N]->[M,N]$，

- 计算时间：$2MKN/FLOPS$
- 访存时间为：$(MN+MK+KN)/memory_bandwidth$，因为首先要**读取**$MK$ 和 NK 这两个矩阵，然后结果还要**写入**MN 这个矩阵里。假设是 fp16，占 2bytes，那就还要乘以 2

假设 $b=1, s=4096, d=d_k=2048$，以计算 Q 和 K 的相似度为例，对比一下训练和预测时的**计算耗时**和**访存耗时**

- **训练时**：$M=4096, K=2048, N=4096$==> **计算是瓶颈**
  - FLOPS：$2\times b\times s^2\times d_k=2\times 1\times 4096^2\times 2048=68719476736$
  - 计算耗时：$FLOPS/max_FLOPS=68719476736/(312\times 10^{12})=0.00022s=220\times 10^{-6}s=220us$
  - 访存耗时：$(MN+MK+KN)/memory_bandwidth=2\times(4096\times 4096+4096\times 2048+2048\times 4096)/(2\times 10^{12})=3.35544\times 10^{-5}s=33.544\times 10^{-6}=33.5544us$
- **预测时**：$M=1, K=2048, N=4096$==> **访存是瓶颈**
  - FLOPS：$2\times b\times 1\times d_k\times s=2\times 1\times 2048\times 4096=16777216$

- 计算耗时：$FLOPS/max_FLOPS = 16777216/(312 \times 10^{12}) = 5.38 \times 10^{-8}s = 0.0538 \times 10^{-6}s = 0.0538us$
- 访存耗时：$(MN + MK + KN)/memory_bandwidth = 2 \times (1 \times 4096 + 1 \times 2048 + 2048 \times 4096)/(2 \times 10^{12}) = 8.3948 \times 10^{-6}s = 8.3948us$

一些常见的名词：

- H2D：host to device，从 cpu 拷贝到 gpu
- D2H：device to host，从 gpu 拷贝到 cpu

1.6 Transformer 加速

1.6.1 lightseq

LightSeq: A High Performance Inference Library for Transformers

LightSeq2: Accelerated Training for Transformer-based Models on GPUs

https://github.com/bytedance/lightseq

2 PLM：仅编码器/仅解码器/编码器 + 解码器

2.1 仅编码器的 BERT

BERT 小学生级上手教程，从原理到上手全有图示，还能直接在线运行

BERT 源码分析（PART I）

BERT 源码分析（PART II）

Dive into BERT：语言模型与知识

关于 BERT，面试官们都怎么问

主要讲了下面 3 篇：

Language Models as Knowledge Bases?

Linguistic Knowledge and Transferability of Contextual Representations

What does BERT learn about the structure of language?

A Primer in BERTology: What we know about how BERT works

摘要：目前，基于 Transformer 的模型已经广泛应用于自然语言处理中，但我们依然对这些模型的内部工作机制知之甚少。在本文中，来自麻省大学洛威尔分校的研究者对流行的 BERT 模型进行综述，并综合分析了 40 多项分析研究。他们还概览了对模型和训练机制提出的改进，然后描画了未来的研究方向。

Pre-trained Models for Natural Language Processing: A Survey

BERT 系列文章汇总导读

ALBERT、XLNet，NLP 技术发展太快，如何才能跟得上节奏?

绝对干货！NLP 预训练模型：从 transformer 到 albert

ALBERT 一作蓝振忠：预训练模型应用已成熟，ChineseGLUE 要对标 GLUE 基准

有哪些令你印象深刻的魔改 Transformer?

BERT 模型超酷炫，上手又太难？请查收这份 BERT 快速入门指南！

2.1.1 multi-head att 实现

输入原始的 query(即 from_tensor) 之后, 把 [batch, from_seq, emb] 变成 [?, emb]，其中?=batch*from_seq

```
from_tensor_2d = reshape_to_matrix(from_tensor)

def reshape_to_matrix(input_tensor):
    """Reshapes a >= rank 2 tensor to a rank 2 tensor (i.e., a matrix)."""
    ndims = input_tensor.shape.ndims
    if ndims < 2:
        raise ValueError("Input tensor must have at least rank 2. Shape = %s" %
            (input_tensor.shape))
    if ndims == 2:
        return input_tensor
    width = input_tensor.shape[-1]
output_tensor = tf.reshape(input_tensor, [-1, width])
    return output_tensor
```

然后再接一个 fc，把 [?, emb] 变成 [?, head_num * per_head]，一般 head_num * per_head=emb。

```
query_layer = tf.layers.dense(
        from_tensor_2d,
        num_attention_heads * size_per_head,
        activation=query_act,
        name="query",
        kernel_initializer=create_initializer(initializer_range))
```

因为?=batch*from_seq，所以可以直接做如下变换

```
query_layer = transpose_for_scores(query_layer, batch_size,
        num_attention_heads, from_seq_length,
        size_per_head)
```

实际就是把? 拆开成 batch, from_seq，整个变成 [batch, from_seq, head_num, per_head]，然后做了个 transpose，把 1 和 2 互换了下，得到 [batch, head_num, from_seq, per_head]

```
def transpose_for_scores(input_tensor, batch_size, num_attention_heads,
        seq_length, width):
    output_tensor = tf.reshape(
            input_tensor, [batch_size, seq_length, num_attention_heads, width])

    output_tensor = tf.transpose(output_tensor, [0, 2, 1, 3])
    return output_tensor
```

然后 key 也做完全一样的操作 (不过处理的是 to_tensor，如果是 self-attention，那 to_tensor=from_tensor), 得到 [batch, head_num, to_seq, per_head]：

```
to_tensor_2d = reshape_to_matrix(to_tensor)
key_layer = tf.layers.dense(
        to_tensor_2d,
        num_attention_heads * size_per_head,
        activation=key_act,
        name="key",
        kernel_initializer=create_initializer(initializer_range))

key_layer = transpose_for_scores(key_layer, batch_size, num_attention_heads,
        to_seq_length, size_per_head)
```

然后就算 QK^T 了，注意这里对 key 取了转置，也就是 [batch, head_num, from_seq, per_head] 乘以 [batch, head_num, per_head, to_seq]，得到的结果是 [batch, head_num, from_seq, to_seq]：

```
attention_scores = tf.matmul(query_layer, key_layer, transpose_b=True)
attention_scores = tf.multiply(attention_scores,
    1.0 / math.sqrt(float(size_per_head)))

if attention_mask is not None:
    # `attention_mask` = [B, 1, F, T]
    attention_mask = tf.expand_dims(attention_mask, axis=[1])

    # Since attention_mask is 1.0 for positions we want to attend and 0.0 for
    # masked positions, this operation will create a tensor which is 0.0 for
    # positions we want to attend and -10000.0 for masked positions.
    adder = (1.0 - tf.cast(attention_mask, tf.float32)) * -10000.0

    # Since we are adding it to the raw scores before the softmax, this is
    # effectively the same as removing these entirely.
    attention_scores += adder
attention_probs = tf.nn.softmax(attention_scores)
attention_probs = dropout(attention_probs, attention_probs_dropout_prob)
```

然后看下 value 的操作：

```
value_layer = tf.layers.dense(
        to_tensor_2d,
        num_attention_heads * size_per_head,
        activation=value_act,
        name="value",
        kernel_initializer=create_initializer(initializer_range))

# `value_layer` = [batch, to_seq, head_num, per_head]
value_layer = tf.reshape(
            value_layer,
            [batch_size, to_seq_length, num_attention_heads, size_per_head])

# `value_layer` = [batch, head_num, to_seq, per_head]
value_layer = tf.transpose(value_layer, [0, 2, 1, 3])

# `context_layer` = [batch, head_num, from_seq, per_head]
context_layer = tf.matmul(attention_probs, value_layer)

# `context_layer` = [batch, from_seq, head_num, per_head]
context_layer = tf.transpose(context_layer, [0, 2, 1, 3])
```

再确认一点，$softmax(QK^T)$ 是 [batch, head_num, from_seq, to_seq]，而 V 是 [batch, head_num, to_seq, per_head]，所以 context_layer 是 [batch, head_num, from_seq, per_head]

最后，再搞一下，变回 [batch, from_seq, head_num * per_head]：

```
if do_return_2d_tensor:
# `context_layer` = [B*F, N*H]
    context_layer = tf.reshape(
        context_layer,
        [batch_size * from_seq_length, num_attention_heads * size_per_head])
else:
```

```
# `context_layer` = [B, F, N*H]
    context_layer = tf.reshape(
        context_layer,
        [batch_size, from_seq_length, num_attention_heads * size_per_head])
```

如上过程是 $Concat(head_1, ..., head_h)$,其中 $head_i = Attention(QW_i^Q, KW_i^K, VW_i^V)$。包装在了函数 `attention_layer` 之中，我们注意到原文还有一个大小为 $hd_v \times d_{model}$ 的 W^O，也就是大小为 $d_{model} \times d_{model}$，再看看源码。。也就是说，正常的 bert 里，`attention_heads` 就只有一个元素，然后接了个 `hidden_size` 的 fc，而前面的代码里也提到了 `hidden_size` 正好就是 d_{model}，所以这就是 W^O。

```
attention_heads = []
with tf.variable_scope("self"):
    attention_head = attention_layer(xxxxx)
    attention_heads.append(attention_head)
    attention_output = None
    if len(attention_heads) == 1:
        attention_output = attention_heads[0]
    else:
        # In the case where we have other sequences, we just concatenate
        # them to the self-attention head before the projection.
        attention_output = tf.concat(attention_heads, axis=-1)
    # Run a linear projection of `hidden_size` then add a residual
    # with `layer_input`.
    with tf.variable_scope("output"):
        attention_output = tf.layers.dense(
            attention_output,
            hidden_size,
            kernel_initializer=create_initializer(initializer_range))
        attention_output = dropout(attention_output, hidden_dropout_prob)
        attention_output = layer_norm(attention_output + layer_input)
```

关于 mask，可以看看这个https://juejin.im/post/5b9f1af0e51d450e425eb32d

摘抄一下：

什么是 padding mask 呢？回想一下，我们的每个批次输入序列长度是不一样的！也就是说，我们要对输入序列进行对齐！具体来说，就是给在较短的序列后面填充 0。因为这些填充的位置，其实是没什么意义的，所以我们的 attention 机制不应该把注意力放在这些位置上，所以我们需要进行一些处理。具体的做法是，把这些位置的值加上一个非常大的负数 (可以是负无穷)，这样的话，经过 softmax，这些位置的概率就会接近 0！

而 sequence mask 是为了使得 decoder 不能看见未来的信息。也就是对于一个序列，在 time_step 为 t 的时刻，我们的解码输出应该只能依赖于 t 时刻之前的输出，而不能依赖 t 之后的输出。因此我们需要想一个办法，把 t 之后的信息给隐藏起来。那么具体怎么做呢？也很简单：产生一个上三角矩阵，上三角的值全为 1，下三角的值权威 0，对角线也是 0。把这个矩阵作用在每一个序列上，就可以达到我们的目的啦。

2.1.2 masked-language-model 的实现

https://github.com/google-research/bert/blob/eedf5716ce1268e56f0a50264a88cafad334ac61/run_pretraining.py#L240

如下，其中 `hidden_size` 就是是 d_{model}：

```
def get_masked_lm_output(bert_config, input_tensor, output_weights, positions,
                         label_ids, label_weights):
  """Get loss and log probs for the masked LM."""
  input_tensor = gather_indexes(input_tensor, positions)

  with tf.variable_scope("cls/predictions"):
    # We apply one more non-linear transformation before the output layer.
    # This matrix is not used after pre-training.
```

```
    with tf.variable_scope("transform"):
      input_tensor = tf.layers.dense(
          input_tensor,
          units=bert_config.hidden_size,
          activation=modeling.get_activation(bert_config.hidden_act),
          kernel_initializer=modeling.create_initializer(
              bert_config.initializer_range))
      input_tensor = modeling.layer_norm(input_tensor)

    # The output weights are the same as the input embeddings, but there is
    # an output-only bias for each token.
    output_bias = tf.get_variable(
        "output_bias",
        shape=[bert_config.vocab_size],
        initializer=tf.zeros_initializer())
    logits = tf.matmul(input_tensor, output_weights, transpose_b=True)
    logits = tf.nn.bias_add(logits, output_bias)
    log_probs = tf.nn.log_softmax(logits, axis=-1)

    label_ids = tf.reshape(label_ids, [-1])
    label_weights = tf.reshape(label_weights, [-1])

    one_hot_labels = tf.one_hot(
        label_ids, depth=bert_config.vocab_size, dtype=tf.float32)

    # The `positions` tensor might be zero-padded (if the sequence is too
    # short to have the maximum number of predictions). The `label_weights`
    # tensor has a value of 1.0 for every real prediction and 0.0 for the
    # padding predictions.
    per_example_loss = -tf.reduce_sum(log_probs * one_hot_labels, axis=[-1])
    numerator = tf.reduce_sum(label_weights * per_example_loss)
    denominator = tf.reduce_sum(label_weights) + 1e-5
    loss = numerator / denominator

  return (loss, per_example_loss, log_probs)
```

其中的 gather 如下:

```
def gather_indexes(sequence_tensor, positions):
  """Gathers the vectors at the specific positions over a minibatch."""
  sequence_shape = modeling.get_shape_list(sequence_tensor, expected_rank=3)
  batch_size = sequence_shape[0]
  seq_length = sequence_shape[1]
  width = sequence_shape[2]

  flat_offsets = tf.reshape(
      tf.range(0, batch_size, dtype=tf.int32) * seq_length, [-1, 1])
  flat_positions = tf.reshape(positions + flat_offsets, [-1])
  flat_sequence_tensor = tf.reshape(sequence_tensor,
                                    [batch_size * seq_length, width])
  output_tensor = tf.gather(flat_sequence_tensor, flat_positions)
  return output_tensor
```

注意调用时传的是如下参数

```
    (masked_lm_loss,
     masked_lm_example_loss, masked_lm_log_probs) = get_masked_lm_output(
         bert_config, model.get_sequence_output(), model.get_embedding_table(),
         masked_lm_positions, masked_lm_ids, masked_lm_weights)
```

2.1.3 BERT 的可解释性

ACL 2019 | 理解 BERT 每一层都学到了什么

What does BERT learn about the structure of language?

探索 BERT 深层次的表征学习是一个非常有必要的事情，一是这可以帮助我们更加清晰地认识 BERT 的局限性，从而改进 BERT 或者搞清楚它的应用范围；二是这有助于探索 BERT 的可解释性

2.2 更复杂的 BERT

站在 BERT 肩膀上的 NLP 新秀们（PART III）

BERT 时代与后时代的 NLP

美团 BERT 的探索和实践

Bert 时代的创新（应用篇）：Bert 在 NLP 各领域的应用进展

2.2.1 中文 BERT

2.2.1.1 WWM

哈工大讯飞联合实验室发布基于全词覆盖的中文 BERT 预训练模型

https://github.com/ymcui/Chinese-BERT-wwm

论文：Pre-Training with Whole Word Masking for Chinese BERT

2.2.1.2 ERNIE

参考中文任务全面超越 BERT：百度正式发布 NLP 预训练模型 ERNIE

ERNIE: Enhanced Representation through Knowledge Integration

使用 entity-level masking 和 phrase-level masking 两种 mask 方法

输入的每个样本由 5 个 ';' 分隔的字段组成，数据格式：

- token_ids
- sentence_type_ids：两句话，第一句都是 0，第二句都是 1
- position_ids
- seg_labels：分词边界信息：0 表示词首、1 表示非词首、-1 为占位符, 其对应的词为 CLS 或者 SEP；
- next_sentence_label

例如：

```
1 1048 492 1333 1361 1051 326 2508 5 1803 1827 98 164 133 2777 2696 983 121 4 19 9 634 551 844 85 14 247
```

和 bert 在 mask 上的区别：

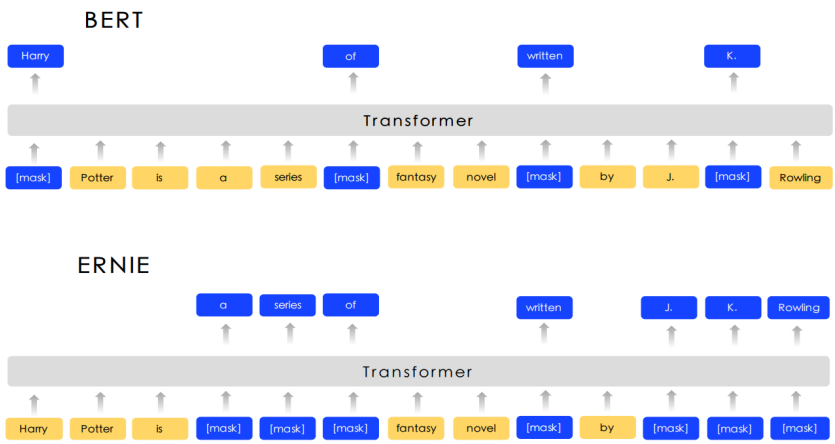


Figure 1: The different masking strategy between BERT and ERNIE

一个句子的不同 level 的 mask 方式：

Sentence	Harry	Potter	is	a	series	of	fantasy	novels	written	by	British	author	J.	K.	Rowling
Basic-level Masking	[mask]	Potter	is	a	series	[mask]	fantasy	novels	[mask]	by	British	author	J.	[mask]	Rowling
Entity-level Masking	Harry	Potter	is	a	series	[mask]	fantasy	novels	[mask]	by	British	author	[mask]	[mask]	[mask]
Phrase-level Masking	Harry	Potter	is	[mask]	[mask]	[mask]	fantasy	novels	[mask]	by	British	author	[mask]	[mask]	[mask]

Figure 2: Different masking level of a sentence

ERNIE 2.0: A CONTINUAL PRE-TRAINING FRAMEWORK FOR LANGUAGE UNDERSTANDING

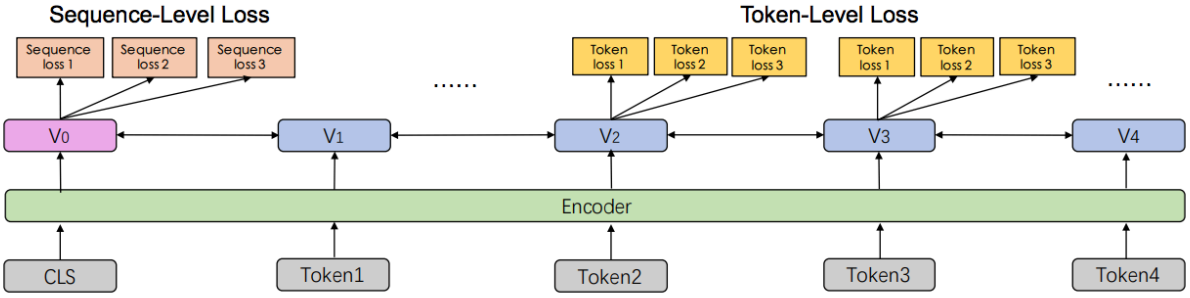


Figure 2: The architecture of multi-task pre-training in the ERNIE 2.0 framework, in which the encoder can be recurrent neural networks or a deep transformer.

2.2.2 跨语言

2.2.2.1 XLM

Massively Multilingual Sentence Embeddings for Zero-Shot Cross-Lingual Transfer and Beyond，XLM 的主要思想还是来自于这篇文章，借用了 BERT 的框架最后成了 XLM。本文提出了 LASER（Language-Agnostic SEntence Representations）

XLM：facebook 提出Cross-lingual Language Model Pretraining，加了 language emb

- 无监督的方法：只依赖单语种数据（monolingual data）
- 有监督的方法：对平行语料使用新的跨语言 loss

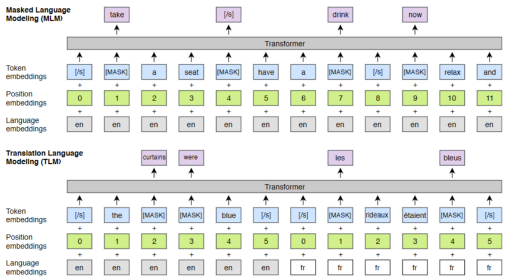


Fig. 5: Cross-lingual language model pretraining. The MLM objective is similar to BERT, but with continuous streams of text as opposed to sentence pairs. The TLM objective extends MLM to pairs of parallel sentences. To predict a masked English word, the model can attend to both the English sentence and its French translation, and is encouraged to align English and French representations. Courtesy of [47].

Facebook 最新语言模型 XLM-R：多项任务刷新 SOTA，超越单语 BERT

XLM-R

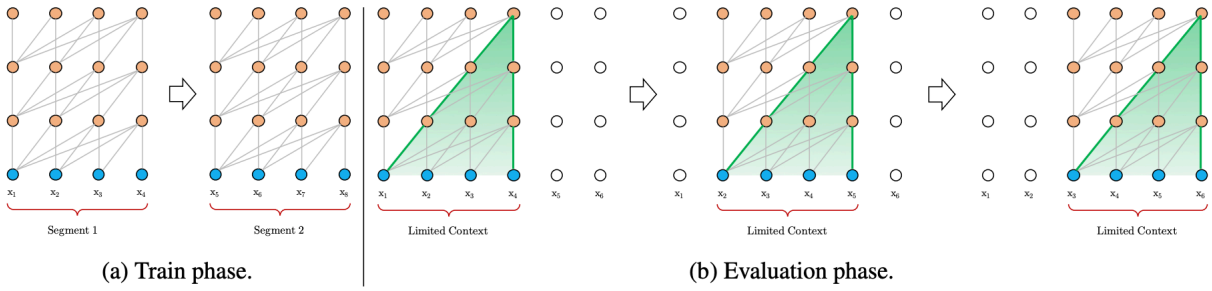
Unsupervised Cross-lingual Representation Learning at Scale

来自 facebook。针对多种跨语言的传输任务，大规模地对多语言语言模型进行预训练可以显著提高性能。在使用超过 2TB 的已过滤 CommonCrawl 数据的基础上，研究者在 100 种语言上训练了基于 Transformer 的掩模语言模型。该模型被称为 XLM-R，在各种跨语言基准测试中，其性能显著优于多语言 BERT（mBERT），其中 XNLI 的平均准确度为 + 13.8%，MLQA 的平均 F1 得分为 + 12.3%，而 FQ 的平均 F1 得分为 + 2.1%NER。XLM-R 在低资源语言上表现特别出色，与以前的 XLM 模型相比，斯瓦希里语（Swahili）的 XNLI 准确性提升了 11.8%，乌尔都语（Urdu）的准确性提升了 9.2%。研究者还对获得这些提升所需的关键因素进行了详细的实证评估，包括（1）积极转移和能力稀释；（2）大规模资源资源的高低性能之间的权衡。最后，他们首次展示了在不牺牲每种语言性能的情况下进行多语言建模的可能性。XLM-Ris 在 GLUE 和 XNLI 基准测试中具有强大的单语言模型，因此非常具有竞争力。

2.2.3 更长序列

2.2.3.1 transformer-xl

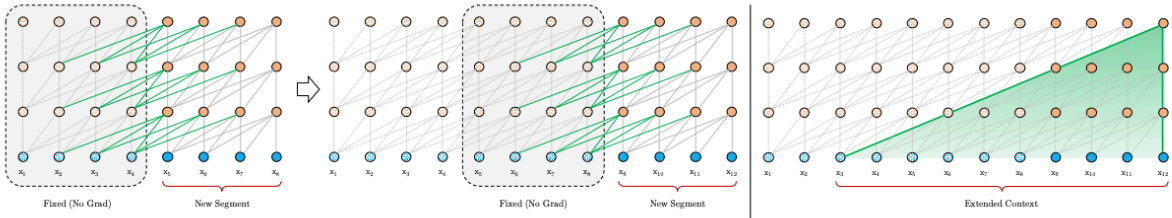
Transformer-XL: Attentive Language Models Beyond a Fixed-Length Context



(a) Train phase. (b) Evaluation phase.

Figure 1: Illustration of the vanilla model with a segment length 4.

最简单的处理长文本方法：对长文本直接切成多个 segment，每个 segment 独立过 transformer，**segment 间是没有信息流动的**



(a) Training phase. (b) Evaluation phase.

Figure 2: Illustration of the Transformer-XL model with a segment length 4.

transformer-xl 的处理方法：参考 RNN 的**隐藏记忆单元**，对**上一个 segment** 计算的隐藏状态序列进行 **fixed 和 cached**，并在模型处理下一个新的 segment 时将其缓存为**可重用的扩展上下文**。

如图，第 k+1 层的第 i 个元素，用到了第 k 层的第 i-1, i-2, …,i - segment_len+1 这几个元素，所以**对于最顶层的来讲，实际看到的窗口就更长了**。为此还提出了**相对位置编码**，即使用两个 token 的**相对距离**代替之前的绝对位置。

此外，Stabilizing Transformers for Reinforcement Learning说明了为什么标准的 transformer 架构很难在强化学习中优化。研究者同时提出了一种架构 Gated Transformer-XL(GTrXL)，可以很好地提升 transformer 架构和变体的稳定性，并加速学习，可以超过 LSTM，在多任务学习 DMLab-30 基准上达到 SOTA 的水平。

2.2.3.2 XLNet

XLNet：运行机制及和 Bert 的异同比较

Transformer-XL 与 XLNet 笔记

什么是 XLNet 中的双流自注意力

20 项任务全面碾压 BERT，CMU 全新 XLNet 预训练模型屠榜（已开源）

参考拆解 XLNet 模型设计，回顾语言表征学习的思想演进

他们创造了横扫 NLP 的 XLNet：专访 CMU 博士杨植麟

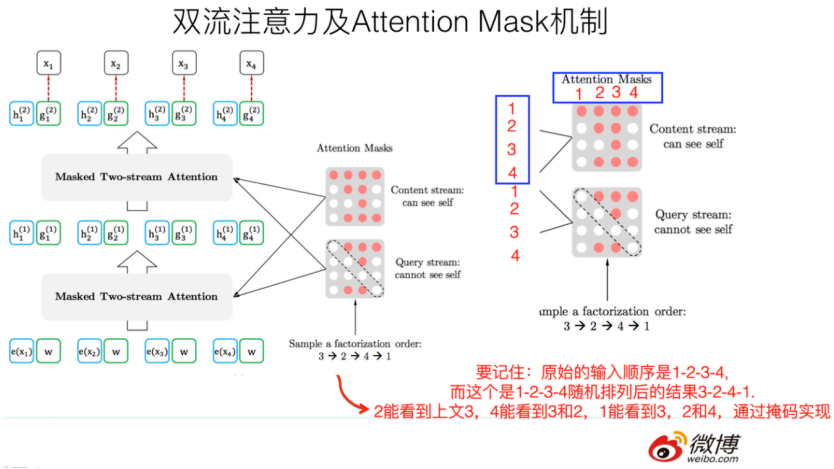
XLNet: Generalized Autoregressive Pretraining for Language Understanding

代码：https://github.com/zihangdai/xlnet

- 自回归（autoregressive, AR）：主要的论文有这几篇：Semi-supervised sequence learning、Deep contextualized word representations、Improving language understanding by generative pre-training。通过一个 autoregressive 的模型来估计文本语料库的概率分布。也就是给定一个文本序列 $\mathbf{x} = (x_1, \cdots, x_T)$，AR 将 likelihood 因式分解 (factorize) 成一个前向的乘积 $p(\mathbf{x}) = \prod_{t=1}^{T} p(x_t|\mathbf{x}_{<t})$，或者是一个后向的乘积 $p(\mathbf{x}) = \prod_{t=T}^{1} p(x_t|\mathbf{x}_{>t})$。由于 AR 语言模型仅被训练用于编码**单向 (uni-directional) 语境（前向或后向）**，因而在深度双向语境建模中效果不佳。而下游语言理解任务通常需要双向语境信息，导致 AR 语言建模无法实现有效预训练。
- 自编码（autoencoding, AE）：相关的预训练模型不会进行明确的密度估计 (explicit density estimation)，而是从残缺的 (corrupted) 输入中**重建原始数据**。例如 bert，使用一定比例的 `[MASK]`，然后预测被 mask 掉的是什么东西。由于目标并不是密度估计，所以在重建的时候，可以考虑双向的上下文信息。但存在如下两个问题：
  - **finetune** 时的真实数据**缺少**预训练期间使用的 `[MASK]` 这些 **mask 信息**，这导致**预训练和微调效果的差异（pretrain-finetune discrepancy）**。
  - 输入中要预测的 token 是被 mask 掉的，所以无法像 AR 那样使用乘积 rule 来建立联合概率分布。也就是说，给定未 mask 的 token，BERT **假设预测的 token** 之间**彼此独立**，这其实是对自然语言中普遍存在的**高阶、长期依赖关系**的一种**过度简化**。

基于这些优缺点，提出了一种泛化的自回归预训练模型 XLNet：

- Permutation Language Model(PLM)：在自回归 LM 模式下，最大化所有可能的**因式分解顺序**的对数似然，学习双向语境信息；
  - 把原来的 `[MASK]` 这个 token 干掉了，转而用 attention mask 来搞
  - 引入了排列：即原来是 1234，可以输入 3241，这个时候改一下 mask 就行
- 引入了 Transformer-XL 的主要思路：**相对位置编码**以及**分段 RNN 机制**，实践已经证明这两点对于长文档任务是很有帮助的



2.2.4 更多的任务

2.2.4.1 MT-DNN

Multi-Task Deep Neural Networks for Natural Language Understanding

2.2.5 其他改进

2.2.5.1 RoBERTa

参考重回榜首的 BERT 改进版开源了，千块 V100、160GB 纯文本的大模型

RoBERTa: A Robustly Optimized BERT Pretraining Approach

- 修改了一些超参
- 删掉 nsp 任务
- 更大的 batchsize 和学习率

RoBERTa 中文预训练模型，你离中文任务的「SOTA」只差个它

https://github.com/brightmart/roberta_zh

2.2.5.2 DeBERTa

Deberta: Decoding-enhanced bert with disentangled attention

- disentangled attention mechanism：每个词使用 2 个向量，H 编码内容，P 编码相对位置，i 和 j 间的 attention 如下：

$$
\begin{aligned}
\mathrm{A}_{i,j} &= \left\{\mathrm{H}_i, \mathrm{P}_{i|j}\right\} \times \left\{\mathrm{H}_j, \mathrm{P}_{j|i}\right\}^{\mathrm{T}} \\
&= \mathrm{H}_i\mathrm{H}_j^{\mathrm{T}} + \mathrm{H}_i\mathrm{P}_{j|i}^{\mathrm{T}} + \mathrm{P}_{i|j}\mathrm{H}_j^{\mathrm{T}} + \mathrm{P}_{i|j}\mathrm{P}_{j|i}^{\mathrm{T}}
\end{aligned}
$$

- 预训练：使用 enhanced mask decoder，把绝对位置信息引入解码层，来预估被 mask 掉的 token
- finetune：virtual adversarial training

2.2.5.3 ELECTRA

2019 最佳预训练模型：非暴力美学，1/4 算力超越 RoBERTa

ELECTRA: 超越 BERT, 19 年最佳 NLP 预训练模型

ELECTRA: pre-training text encoders as discriminators rather than generators

使用新的预训练 task：RTD(replaced token detection)：

- 使用一个**生成网络**采样出 token，替换一些原有的 token
- 训练一个**判别模型**，预估一个 token **是否是被替换的**

RTD 比 MLM 更 **sample-efficient**，因为 RTD 只做二分类，而 MLM 需要做全词表的 softmax

2.2.5.4 Matryoshka

俄罗斯套娃 (Matryoshka) 嵌入模型概述

Matryoshka representation learning

俄罗斯套娃嵌入模型旨在将**更重要的信息存储在早期的维度中**，将不太重要的信息存储在后面的维度中。俄罗斯套娃嵌入模型的这一特点允许我们截断模型产生的原始 (大) 嵌入，同时仍保留足够的信息以在下游任务上表现良好。

基本原理：假设是一个 L 分类的多分类问题，把原来的 x 先通过 F 映射到 d，然后取前 m 维出来，再通过 W 把 m 映射到 L，去和 label 算 loss，再乘以权重 c。这么搞几次，相当于搞了多个 mlp，把不同的 m 映射到 L。

假设输入一个 llm 产出的 4096 维向量，那这个 llm 就是 F，假设是一个 2 分类问题，期望降维到 (64,128,1024)，那就是取前 64 出来，映射成 2 维，算个 loss，再取前 128 维出来，映射成 2 维，算个 loss，再取前 1024 出来，算个 loss

但这样比较粗暴，合理的方式是在 gpt 的建议下，通过如下方式来循序渐进地完成降维：

```
multi_stage_cfg = [
    {"stage": "stage1_high_dim", "dims": [4096, 2048], "weights": [0.7, 0.3]},
    {"stage": "stage2_mid_dim", "dims": [2048, 1024, 512], "weights": [0.5, 0.3, 0.2]},
    {"stage": "stage3_low_dim", "dims": [512, 128, 64], "weights": [0.2, 0.3, 0.5]},
]

def train_stage(model, mrl_dims, mrl_weights, output_dir,
    train_dataset, eval_dataset, lr=5e-6, bs=32, acc=2,
    max_steps=None, stage_name=""):

    if wandb.run:
        wandb.finish()
    if rank == 0:
        wandb.init(project="xxx", name=stage_name)
    loss_fn = losses.MultipleNegativesRankingLoss(model)
    loss = losses.MatryoshkaLoss(model, loss_fn,
        matryoshka_dims=mrl_dims, matryoshka_weights=mrl_weights)

    trainer = SentenceTransformerTrainer(
        model=model,
        args=args,
        train_dataset=train_dataset,
        eval_dataset=eval_dataset,
        loss=loss,
    )
    trainer.train()
    # 其他

def train():
    for stage_cfg in multi_stage_cfg:
        if stage_cfg["stage"] == "stage1_high_dim":
            model = load_model(base_model_path)
        stage_dir = f"./outputs/{stage_cfg['stage']}"
        print(f"\n Training {stage_cfg['stage']}... dims={stage_cfg['dims']}")
        train_stage(
            model=model,
```

```
            mrl_dims=stage_cfg["dims"],
            mrl_weights=stage_cfg["weights"],
            output_dir=stage_dir,
            stage_name=stage_cfg["stage"],
            train_dataset=train_dataset,
            eval_dataset=eval_dataset,
            lr=5e-6,
            bs=bs,
            acc=acc,
            max_steps=max_steps
        )
        torch.cuda.empty_cache()
        torch.cuda.synchronize()
        gc.collect()
        model = load_model_continue(stage_dir)
```

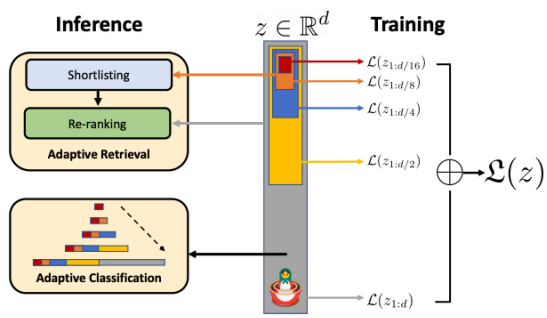


Figure 1: Matryoshka Representation Learning is adaptable to any representation learning setup and begets a Matryoshka Representation z by optimizing the original loss $\mathcal{L}(.)$ at $O(\log(d))$ chosen representation sizes. Matryoshka Representation can be utilized effectively for adaptive deployment across environments and downstream tasks.

https://sbert.net/examples/training/matryoshka/README.html

训练 + 预测

```
model_name = "./bge-large-zh-v1.5"
out_model_name = "./new-bge-large-zh-v1.5"
model = SentenceTransformer(model_name)


sentences = [" 这是一个测试句子", " 这是另一个例子"]
embeddings = model.encode(sentences)
print(f" 嵌入维度: {embeddings.shape}")

train_dataset = Dataset.from_dict({
    "anchor": ["It's nice weather outside today.", "He drove to work."],
    "positive": ["It's so sunny.", "He took the car to the office."],
})
loss = losses.MultipleNegativesRankingLoss(model)
loss = losses.MatryoshkaLoss(model, loss, [768, 512, 256, 128, 64])

trainer = SentenceTransformerTrainer(
    model=model,
```

```
    train_dataset=train_dataset,
    loss=loss,
)
trainer.train()
trainer.save_model(out_model_name)

model = SentenceTransformer(out_model_name)

matryoshka_dim = 64
embeddings = model.encode(
            [
                "The weather is so nice!",
                "It's so sunny outside!",
                "He drove to the stadium.",
                ]
            )
embeddings = embeddings[..., :matryoshka_dim] # Shrink the embedding dimensions
print(embeddings.shape)
```

注意，如果嵌入已经归一化，那么在截断后它们将不再归一化，因此你**可能需要重新归一化**。

```
from sentence_transformers import SentenceTransformer
from sentence_transformers.util import cos_sim
import torch.nn.functional as F

model = SentenceTransformer("nomic-ai/nomic-embed-text-v1.5", trust_remote_code=True)

matryoshka_dim = 64
embeddings = model.encode(
    [
        "search_query: What is TSNE?",
        "search_document: t-distributed stochastic xxx.",
        "search_document: Amelia Mary Earhart was xxx.",
    ],
    convert_to_tensor=True,
)
# The Nomic team uses a custom architecture,
# making them recommend Layer Normalization before truncation
embeddings = F.layer_norm(embeddings, normalized_shape=(embeddings.shape[1],))
embeddings[..., :matryoshka_dim] # Shrink the embedding dimensions

similarities = cos_sim(embeddings[0], embeddings[1:])
# => tensor([[0.7154, 0.4468]])
```

2.2.5.5 Piccolo2

拿下 SOTA！最强中文 Embedding 模型对标 OpenAI，技术路线公开

Piccolo2: General Text Embedding with Multi-task Hybrid Loss Training

https://huggingface.co/sensenova/piccolo-large-zh-v2

2.2.5.6 Conan-Embedding

通过负样本挖掘炼出更强 Embedding 模型

Conan-embedding: General Text Embedding with More and Better Negative Samples

2.2.5.7 MEXMA

MEXMA: Token-level objectives improve sentence representations

https://huggingface.co/facebook/MEXMA

https://github.com/facebookresearch/mexma

2.2.5.8 ModernBert

时隔 6 年，谷歌 BERT 终于有替代品了！更快更准更长，还不炒作 GenAI

Smarter, Better, Faster, Longer: A Modern Bidirectional Encoder for Fast, Memory Efficient, and Long Context Finetuning and Inference

2.3 更小的 BERT

BERT 瘦身之路：Distillation，Quantization，Pruning

2.3.1 albert

刚刚，Google 发布 24 个小型 BERT 模型，直接通过 MLM 损失进行预训练

ALBERT：用于语言表征自监督学习的轻量级 BERT

谷歌 ALBERT 模型 V2+ 中文版来了：之前刷新 NLP 各大基准，现在 GitHub 热榜第二

超小型 BERT 中文版横空出世！模型只有 16M，训练速度提升 10 倍

https://github.com/brightmart/albert_zh

预训练小模型也能拿下 13 项 NLP 任务，谷歌 ALBERT 三大改造登顶 GLUE 基准

ALBERT 模型在 GLUE、RACE 和 SQuAD 基准测试上都取得了新的 SOTA 效果，并且参数量还少于 BERT-large。

ALBERT: a lite bert for self-supervised learning of language representations

通过对词嵌入矩阵进行因式分解，再为下游任务共享不同层的所有参数，这样可以大大降低 BERT 的参数量。

还提出了一种新型句间连贯性损失函数，它可以强迫模型学习句间的连贯性表达，从而有利于各种下游 NLP 任务。

ALBERT 通过两个参数削减技术克服了扩展预训练模型面临的主要障碍。

- 对嵌入参数化进行因式分解：将大的嵌入矩阵分解为两个小的矩阵，从而将隐藏层的大小与词汇嵌入的大小分离开来。这种分离使得隐藏层的增加更加容易，同时不显著增加词汇嵌入的参数量。
- 跨层参数共享：避免参数量随着网络深度的增加而增加。

两种技术都显著降低了 BERT 的参数量，同时不对其性能造成明显影响，从而提升了参数效率。ALBERT 的配置类似于 BERT-large，但参数量仅为后者的 1/18，训练速度却是后者的 1.7 倍。这些参数削减技术还可以充当某种形式的正则化，可以使训练更加稳定，而且有利于泛化。

为了进一步提升 ALBERT 的性能，还引入了一个自监督损失函数，用于句子级别的预测（SOP）。SOP 主要聚焦于**句间连贯**，用于解决原版 BERT 中下一句预测（NSP）损失低效的问题。

albert_tiny：

input_ids 先查 word_embeddings(\(V\times E=21118*128))，得到 dim=128 的表示，再查 word_embeddings_2(\(E\times M =128*312\))，得到 dim=312 的表示。

搞 positionembedding 时，并不用输入 0 1 2…，只需要做一些 slice 的变换就行了

```
  with tf.control_dependencies([assert_op]):
    full_position_embeddings = tf.get_variable(
        name=position_embedding_name,
        shape=[max_position_embeddings, width],
        initializer=create_initializer(initializer_range))
    # Since the position embedding table is a learned variable, we create it
    # using a (long) sequence length `max_position_embeddings`. The actual
    # sequence length might be shorter than this, for faster training of
    # tasks that do not have long sequences.
    #
    # So `full_position_embeddings` is effectively an embedding table
    # for position [0, 1, 2, ..., max_position_embeddings-1], and the current
    # sequence has positions [0, 1, 2, ... seq_length-1], so we can just
    # perform a slice.
    position_embeddings = tf.slice(full_position_embeddings, [0, 0],
                                   [seq_length, -1])
    num_dims = len(output.shape.as_list())

    # Only the last two dimensions are relevant (`seq_length` and `width`), so
    # we broadcast among the first dimensions, which is typically just
    # the batch size.
    position_broadcast_shape = []
    for _ in range(num_dims - 2):
      position_broadcast_shape.append(1)
    position_broadcast_shape.extend([seq_length, width])
    position_embeddings = tf.reshape(position_embeddings,
                                     position_broadcast_shape)
    output += position_embeddings
```

然后会通过 create_attention_mask_from_input_mask 把 input_ids 和 input_mask 搞一下，得到 attention_mask 去和 attention 做 mask，主要是算 loss 啥的，把后面的 mask 掉不算

2.3.2 distillbert

参考小版 BERT 也能出奇迹：最火的预训练语言库探索小巧之路

1.4w 个 stars。。

https://huggingface.co/transformers

DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter

2.3.3 tinybert

TinyBERT：模型小 7 倍，速度快 8 倍，华中科大、华为出品

TinyBERT: Distilling BERT for Natural Language Understanding

提出了一个 two-stage learning framework，在 pre-training 阶段和 task-specific 阶段都进行 distill。

相比 baseline，只有 28% parameters 和 31% 的 inference 时间

在 glue 上，7.5x 小，infer 上有 9.4x 快。

哪吒”出世！华为开源中文版 BERT 模型

NEZHA: Neural Contextualized Representation for Chinese Language Understanding

https://github.com/huawei-noah/Pretrained-Language-Model

华为诺亚方舟开源哪吒、TinyBERT 模型，可直接下载使用

2.3.4 reformer

哈希革新 Transformer：这篇 ICLR 高分论文让一块 GPU 处理 64K 长度序列

Reformer: The Efficient Transformer

https://github.com/google/trax/blob/master/trax/models/research/reformer.py

大型的 Transformer 往往可以在许多任务上实现 sota，但训练这些模型的成本很高，尤其是在序列较长的时候。在 ICLR 的入选论文中，我们发现了一篇由谷歌和伯克利研究者发表的优质论文。文章介绍了两种提高 Transformer 效率的技术，最终的 Reformer 模型和 Transformer 模型在性能上表现相似，并且在长序列中拥有更高的存储效率和更快的速度。论文最终获得了「8，8，6」的高分。在最开始，文章提出了将点乘注意力（dot-product attention）替换为一个使用局部敏感哈希（locality-sensitive hashing）的点乘注意力，将复杂度从 O(L2) 变为 O(L log L)，此处 L 指序列的长度。此外，研究者使用可逆残差（reversible residual layers）代替标准残差（standard residuals），这使得存储在训练过程中仅激活一次，而不是 n 次（此处 n 指层数）。最终的 Reformer 模型和 Transformer 模型在性能上表现相同，同时在长序列中拥有更高的存储效率和更快的速度。

大幅减少 GPU 显存占用：可逆残差网络 (The Reversible Residual Network)

2.3.5 LTD-bert

内存用量 1/20，速度加快 80 倍，腾讯 QQ 提出全新 BERT 蒸馏框架，未来将开源

2.3.6 Q-bert

AAAI 2020 | 超低精度量化 BERT，UC 伯克利提出用二阶信息压缩神经网络

Q-BERT: Hessian Based Ultra Low Precision Quantization of BERT

2.3.7 Adabert

推理速度提升 29 倍，参数少 1/10，阿里提出 AdaBERT 压缩方法

AdaBERT: Task-Adaptive BERT Compression with Differentiable Neural Architecture Search

2.4 仅解码器的 GPT

AI 也能精彩表达：几种经典文本生成模型一览

2.4.1 GPT

2018 年的Improving language understanding by generative pre-training，生成式预训练（Generative pre-training, gpt），用 transformer 的 decoder，参数量 117m（0.1b），无监督预训练和有监督微调。

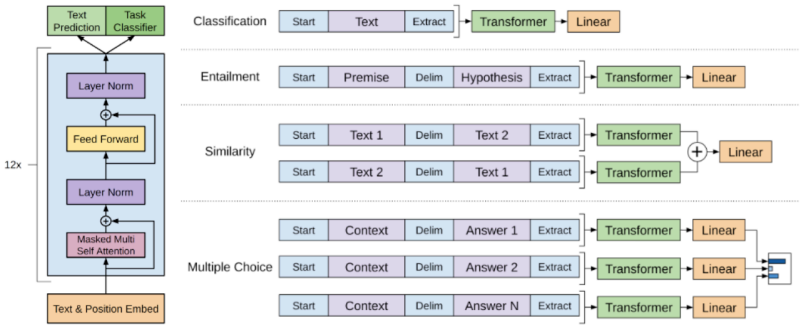


Fig. 7: High-level overview of GPT pretraining, and fine-tuning steps. Courtesy of OpenAI.

微调阶段为每种下游任务专门设计：

- 分类：输入一段文本，经过 transformer，最后接一个 Linear
- entailment(蕴含)：输入 2 段文本，premise(假设) 和 hypothesis(假说)，经过 transformer，最后接一个 Linear
- 相似度：输入 2 段文本 a 和 b，a+b 过 transformer，b+a 过 transformer，再合起来接一个 Linear
- 多选题：输入 context+ 答案 1 过 transformer+Linear，答案 2、答案 3 同样操作，将 3 个输出合在一起求 softmax

2.4.2 GPT2

2019 年的Language models are unsupervised multitask learners模型结构小改，增加数据，参数量变大为 15 亿（1.5b），无监督语言建模。

- layernorm 前移到每个 sub-block 之前
- additional layernorm 在最后的 self-attention block 才加上
- 修改初始化方法，以考虑残差路径上的累积并缩放残差层的权重

15 亿参数最强通用 NLP 模型面世！Open AI GPT-2 可信度高于所有小模型

中文 GPT2

只需单击三次，让中文 GPT-2 为你生成定制故事

https://github.com/imcaspar/gpt2-ml

https://colab.research.google.com/github/imcaspar/gpt2-ml/blob/master/pretrained_model_demo.ipynb

语言模型秒变 API，一文了解如何部署 DistilGPT-2

huggingface 的 distill gpt-2：https://github.com/huggingface/transformers

2.4.3 nanogpt 代码解读

简化版的 gpt，tiktoken：gpt2 中使用的开源分词工具，比 huggingface 的 tokenizer 快得多

```
import tiktoken
enc = tiktoken.get_encoding("gpt2")

# 字节对编码过程，我的输出是 [31373, 995]
encoding_res = enc.encode("hello world")
print(encoding_res)

# 字节对解码过程，解码结果：hello world
```

```
raw_text = enc.decode(encoding_res)
print(raw_text)
```

类似的：https://github.com/karpathy/llm.c

https://github.com/karpathy/build-nanogpt/tree/master

fork 了一个：https://github.com/daiwk/build-nanogpt

GPT 的整体结构：

```
class GPT(nn.Module):

    def __init__(self, config):
        super().__init__()
        self.config = config

        self.transformer = nn.ModuleDict(dict(
            wte = nn.Embedding(config.vocab_size, config.n_embd),
            wpe = nn.Embedding(config.block_size, config.n_embd),
            # 这里有 n 层，每层是一个 Block
            h = nn.ModuleList([Block(config) for _ in range(config.n_layer)]),
            # 最后一层需要加一个 layer norm
            ln_f = nn.LayerNorm(config.n_embd),
        ))
        # 最后一层的 linear
        self.lm_head = nn.Linear(config.n_embd, config.vocab_size, bias=False)

        # weight sharing scheme
        self.transformer.wte.weight = self.lm_head.weight

        # init params
        self.apply(self._init_weights)
```

```
class Block(nn.Module):

    def __init__(self, config):
        super().__init__()
        self.ln_1 = nn.LayerNorm(config.n_embd)
        self.attn = CausalSelfAttention(config)
        self.ln_2 = nn.LayerNorm(config.n_embd)
        self.mlp = MLP(config)

    def forward(self, x):
        # 这里对原来的 add&norm 进行了改造，保留一个从始至终的 +x 的操作，让梯度能够直通输入
        x = x + self.attn(self.ln_1(x))
        x = x + self.mlp(self.ln_2(x))
        return x
```

```
class MLP(nn.Module):

    def __init__(self, config):
        super().__init__()
        self.c_fc    = nn.Linear(config.n_embd, 4 * config.n_embd)
        self.gelu    = nn.GELU(approximate='tanh')
        self.c_proj  = nn.Linear(4 * config.n_embd, config.n_embd)
```

```
        self.c_proj.NANOGPT_SCALE_INIT = 1

    def forward(self, x):
        x = self.c_fc(x)
        x = self.gelu(x)
        x = self.c_proj(x)
        return x
```

```
class CausalSelfAttention(nn.Module):

    def __init__(self, config):
        super().__init__()
        assert config.n_embd % config.n_head == 0
        # key, query, value projections for all heads, but in a batch
        self.c_attn = nn.Linear(config.n_embd, 3 * config.n_embd)
        # output projection
        self.c_proj = nn.Linear(config.n_embd, config.n_embd)
        self.c_proj.NANOGPT_SCALE_INIT = 1
        # regularization
        self.n_head = config.n_head
        self.n_embd = config.n_embd
        # not really a 'bias', more of a mask, but following the OpenAI/HF naming though
        self.register_buffer("bias", torch.tril(torch.ones(config.block_size, config.block_size))
                                     .view(1, 1, config.block_size, config.block_size))

    def forward(self, x):
        B, T, C = x.size() # batch size, sequence length, embedding dimensionality (n_embd)
        # calculate query, key, values for all heads in batch and move head forward to be the batch dim
        # nh is "number of heads", hs is "head size", and C (number of channels) = nh * hs
        # e.g. in GPT-2 (124M), n_head=12, hs=64, so nh*hs=C=768 channels in the Transformer
        qkv = self.c_attn(x)
        q, k, v = qkv.split(self.n_embd, dim=2)
        k = k.view(B, T, self.n_head, C // self.n_head).transpose(1, 2) # (B, nh, T, hs)
        q = q.view(B, T, self.n_head, C // self.n_head).transpose(1, 2) # (B, nh, T, hs)
        v = v.view(B, T, self.n_head, C // self.n_head).transpose(1, 2) # (B, nh, T, hs)
        y = F.scaled_dot_product_attention(q, k, v, is_causal=True) # flash attention
        y = y.transpose(1, 2).contiguous().view(B, T, C) # re-assemble all head outputs side by side
        # output projection
        y = self.c_proj(y)
        return y
```

2.5 编码器 + 解码器

2.5.1 T5

谷歌 T5 模型刷新 GLUE 榜单，110 亿参数量，17 项 NLP 任务新 SOTA

谷歌最新 T5 模型 17 项 NLP 任务霸榜 SuperGLUE，110 亿参数量！

Exploring the limits of transfer learning with a unified text-to-text transformer

将所有 NLP 任务都建模成 text-to-text 的生成任务，

mT5（mt5: A massively multilingual pre-trained text-to-text transformer）是 T5 的变种，基于新的 Common Crawl 的数据集（包括 101 种语言）上预训练

2.5.2 MASS

MASS: Masked Sequence to Sequence Pre-training for Language Generation

bert 只使用了 Transformer 的 encoder 部分，其下游任务也主要是适用于自然语言理解（NLU），对于类似文本摘要、机器翻译、对话应答生成等自然语言生成（NLG）任务显然是不太合适的。MASS 采用了编码器-解码器框架，并尝试在**给定部分句子**的情况下**修复整个句子**。MASS 输入句子包含了一些连续的 Token，并且中间会带有一些**连续的 Mask**，模型的任务是**预测出被 Mask 掉的词**是什么。相比 BERT 只有编码器，MASS 联合训练编码器与解码器，能获得更适合机器翻译的表征能力。

训练步骤主要分为两步：

- Encoder：输入为被随机 mask 掉连续部分 token 的句子，使用 Transformer 对其进行编码；这样处理的目的是可以使得 encoder 可以更好地捕获没有被 mask 掉词语信息用于后续 decoder 的预测；
- Decoder：输入为与 encoder 同样的句子，但是 mask 掉的正好和 encoder 相反，和翻译一样，使用 attention 机制去训练，但只预测 encoder 端被 mask 掉的词。该操作可以迫使 decoder 预测的时候更依赖于 source 端的输入而不是前面预测出的 token，防止误差传递。

2.5.3 BART

多项 NLP 任务新 SOTA，Facebook 提出预训练模型 BART

BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension

BART 是一个适用于 seq2seq 模型的去噪自编码器。预训练包括两个阶段：

- 使用**任意噪声**函数**破坏文本**
- 用 seq2seq 模型来**重建原始文本**